



FLIP THROUGH BEFORE YOU DOWNLOAD

Test It Like a User

Three real pages, straight off the shipping files,
plus everything the full download includes.



The week at a glance

Day	The class does	You do
1 Watch	Give one person a task and watch without helping. Write down what they <i>did</i> , not what they said.	Model it badly first — helping, explaining, leading — then well.
2 Test	Three testers each, the same three tasks, silence. Findings on paper.	Enforce the one rule: if you had to explain it, that is a finding.
3 Sort	Group the findings. Decide which are real problems and which are one person's preference.	Push on "everybody said" when two people said it.
4 Choose	Pick three fixes out of everything found, and write the evidence for each one.	Refuse any fix with no observation behind it.
5 Fix and re-test	Watch the fix happen, then test it again on somebody who has not seen it.	Prompt on the board: make them test it.

SNEAK PEEK · 24 pages + the app

Day 5 is the one that makes the difference, rather than an activity.

Almost nobody knows that fixes something and then watches it still confuse somebody has learned the most lesson, which is that *a fix is a hypothesis*, not an ending.

How long it really takes

Days 1, 3 and 4 are a lesson each, and day 1 can be as short as twenty minutes. **Day 2 is as long as you can give it** — three testers each takes about twenty-five minutes once the class knows the rules. **Day 5 is thirty minutes**, split roughly in half between the fix and the re-test. The days do not have to be consecutive and the unit works better spread across two weeks.

What you need

One computer you can project. Any AI chat tool you already use, for day 5 only. Printed findings sheets. A handful of devices for day 2, or one device and a line of testers. And the app in this download, which opens by double-click and needs nothing installed.

The six flaws, and what a tester will say

Read this, then put it away. **Knowing the answers is what lets you tell whether your class is finding real things or guessing**, and lets you steer a task toward a flaw nobody has hit yet.

#	What is wrong	What a tester tends to say
1	The big Submit button only saves a draft. A second, smaller button called Confirm booking is the one that actually books.	"I thought I'd booked it."
2	Nothing says it worked. After a real booking the screen looks almost identical — a four-character code appears in small gray text and that is the whole confirmation.	"Did that work?"
3	Every error message is the single word Error , whether the slot is taken, the day is missing or the code is wrong.	"It won't let me and it won't say why."
4	The steps only work in one order. Pick the day first, then the slot. Picking a slot and then changing the day silently clears the slot, and nothing says so.	"What I picked."
5	Book the whole day is the biggest button on the screen and is almost never what anyone wants. Cancel a booking is the smallest thing on the book screen.	"I nearly booked the whole day by accident."
6	The confirmation code is shown once, is wiped by the next thing you do, and is required on a different screen in order to cancel.	"Nobody told me to write that down."

SNEAK PEEK · 24 pages + the app

If the class has found four of these by the end of day 2, the testing is working.

If they have found one, the problem is almost always that they are helping. Go back to day 1 for five minutes and model it badly again.

Steering without telling

Flaws 1, 2 and 4 come out of task one. Flaw 6 and flaw 3 come out of task two. Flaw 5 comes out of task three, or out of anybody who is in a hurry. **If a flaw is going unfound, add a tester and give them that task** rather than announcing it.

The prompts, written out in full

You are at the keyboard. Open your AI chat tool, **attach or paste the whole** `makerspace-booker.html` file, and use these in order. The parts in yellow are the only parts you change.

PROMPT 1 • THE FIX

This is a single-file HTML app that books slots in a room.
Make exactly ONE change and nothing else:

`<paste the fix, in the class's own words>`

Rules:

- It must stay one HTML file that opens by double-click and works offline.
- Do not add any field that asks for a person's name.
- Do not change anything I did not ask you to change.
- Give me back the whole file, not a snippet.

PROMPT 2 • WHEN IT DID SOMETHING ELSE AS WELL

You changed more than I asked. Start with the code I gave you
and make only this change:
Leave every other bit of code and word exactly as it was.

SNEAK PEEK • 24 pages + the app

PROMPT 3 • WHEN THE NEW VERSION IS BROKEN

That version does not work. When I `<what you did>`,
`<what happened instead>`.
Go back to the version before your last change and fix only that.

Worked example, for the fix most classes choose

If the class picks the missing confirmation, the middle of prompt 1 reads: "When a booking is made, show a green message under the buttons that says Booked, followed by the day and the time. Keep showing it until the next thing the user does." That is what a well-written fix looks like — a stranger could do it, and you can tell whether it worked by looking.

The sentence to say while it is generating.

"If this comes back different from what we asked, **we are right and it is wrong.**" Children assume the computer knows better. Day 5 is the only chance all week to say otherwise while they watch.



WHAT IS INSIDE

Test It Like a User

- ✓ **The app is included - and flawed on purpose**
- ✓ All six flaws listed for you, with what a tester says
- ✓ **The three tester tasks**, written out word for word
- ✓ Six student pages worth printing
- ✓ **Day 5: fix one live, then re-test on somebody new**
- ✓ What to do when the AI breaks it in front of the class

Running it with your whole staff? Additional teacher licenses are just \$9, and we accept purchase orders through TpT for Schools.

